Experiment 7

Seven-Segment Display Multiplexing, Counters and a Stopwatch

Student Seyed Kiarash Eshraghi Naeini (403101383)
Semester Fall 2025 session: 27–29 December 2025
Platform Zynq-7010 (XC7Z010-CLG400-1), 50 MHz clock, Vivado 2019.1

Summary

This experiment drives the four-digit seven-segment display of the lab board. Starting from a static hexadecimal display, it introduces clock division, time-multiplexing of the digits, a scrolling text display and finally a stopwatch built from cascaded modulo counters (0.1 s resolution, up to 9 min 59.9 s). Three bonus designs extend the stopwatch with speed control and start/stop/reset buttons and animate a "snake" around the display. Every design was re-verified for this report with a self-checking Icarus Verilog testbench (9/9 pass) and re-synthesised with Yosys; the Vivado implementation reports of the lab session show all timing constraints met. Two display errors found during verification (Parts 2 and 5) are documented and corrected in Section 10. Source code, testbenches and constraint files: `https://github.com/seyedkiarash/seyedkiarash.github.io/tree/main/labs/logic-circuits-lab/lab07-seven-segment`.

1 Hardware and conventions

- **Clock.** A 50 MHz oscillator on pin U18 (`create_clock -period 20`).
- **Display.** Four digits share the seven segment lines `a`–`g`; `dig[3:0]` selects the digits, `dig[3]` being the leftmost. The segments are *active-low* (a 0 lights a segment), the digit enables *active-high*.
- **Decoder.** `Hex_to_7_seg` maps a 4-bit value to the segment pattern of `0`–`9`, `A`, `b`, `C`, `d`, `E`, `F`.
- **Clock divider.** A counter toggles its output every H input cycles, so $f_{\text{out}} = f_{\text{clk}}/(2H)$. The values used in this experiment are listed in Table 1.

Table 1. Clock divider settings ($f_{\text{clk}} = 50\,\text{MHz}$).

Use	Half-period H (cycles)	f_{out}	Period
Digit scanning (Parts 3, 5, 6, bonuses)	12,500	2 kHz	0.5 ms per digit
Stopwatch tick (Part 6)	2,500,000	10 Hz	0.1 s
Stopwatch tick (Bonus A initial, Bonus B)	5,000,000	5 Hz	0.2 s
Student-number step (Part 2)	12,500,000	2 Hz	0.5 s
Moving digit / scrolling (Parts 4, 5)	50,000,000	0.5 Hz	2 s

2 Part 1 – Static hexadecimal display

`DIP[3:0]` is decoded and all four digit enables are driven high. Because the digits share the segment lines, every digit shows the same character (Fig. 1). The testbench applies all 256 DIP settings and compares the segments with an independent reference table: 0 errors.



Figure 1. Part 1: with all digits enabled the same character appears four times.

3 Part 2 – Student number, one digit at a time

A 2 Hz clock advances a state counter (0–9); a case statement maps each state to one digit of the student number 403101383, shown on all four digits. State 9 shows `0` and acts as a short pause before the sequence repeats. The testbench (step divider shortened to 4 cycles) records the displayed sequence `4031013830`, as expected.

4 Part 3 – Time-multiplexing

To show different characters on the four digits with only one set of segment lines, the digits are enabled one at a time and the segment lines carry the character of the active digit. A 2 kHz scan clock steps a 2-bit counter; each digit is lit for 0.5 ms and refreshed every 2 ms (500 Hz), far above the ≈60 Hz at which flicker becomes visible, so the eye sees a steady `1234` (Fig. 2). Multiplexing needs $7 + 4 = 11$ outputs instead of $4 \times 7 = 28$.

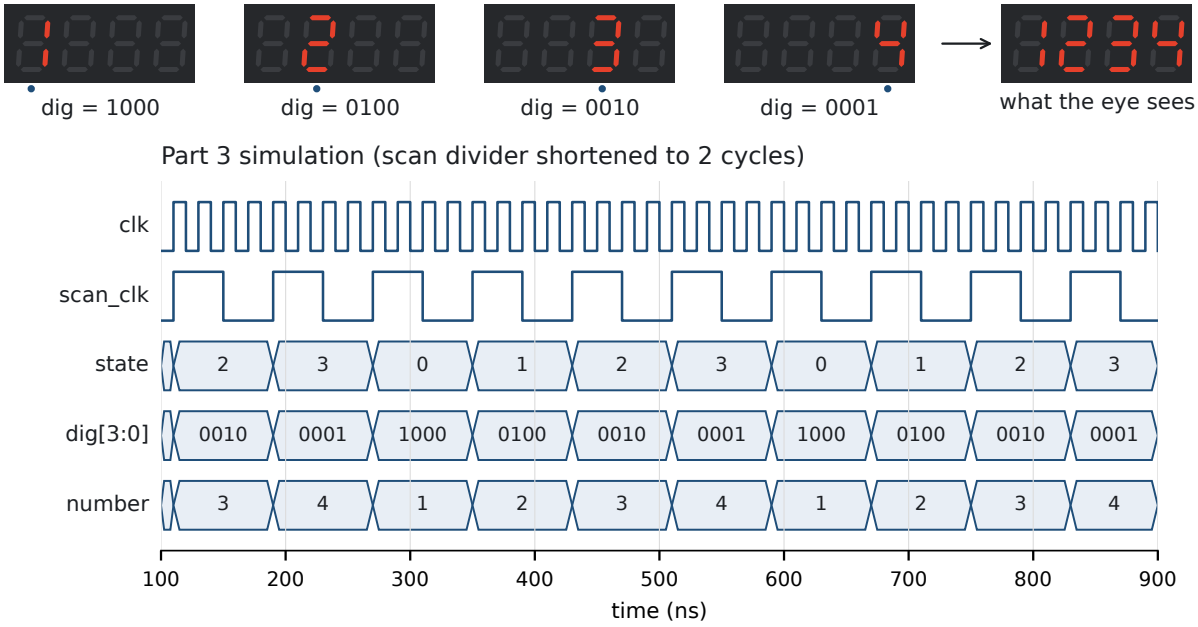


Figure 2. Part 3: the four scan states and the perceived image (top); simulation with the scan divider shortened to 2 cycles (bottom) – `dig` rotates `1000`→`0100`→`0010`→`0001` together with the digit value.

The testbench reconstructs the image from the enable and segment signals and checks that exactly one digit is enabled at any time: frame `"1234"`, 0 one-hot violations.

5 Part 4 – A moving digit

The same rotating enable as in Part 3, but driven by a 0.5 Hz clock and with a constant value: the digit `3` jumps one position to the right every 2 s. Running the rotation slowly makes the mechanism of Part 3 visible. The testbench checks that the character is always `3` and that each move goes exactly one position to the right (25 moves, 0 errors).

6 Part 5 – Scrolling student number

Two clocks are derived from the board clock: the 2 kHz scan clock of Part 3 and a 0.5 Hz shift clock that selects one of nine 4-digit windows over the student number (with wrap-around), so the number scrolls across the display (Fig. 3). The window is held in a 16-bit register `numbers_state` whose top nibble is the first (leftmost) digit.

The testbench shortens both dividers, reconstructs the full display image in every window and compares it with the expected text: 9/9 windows correct.

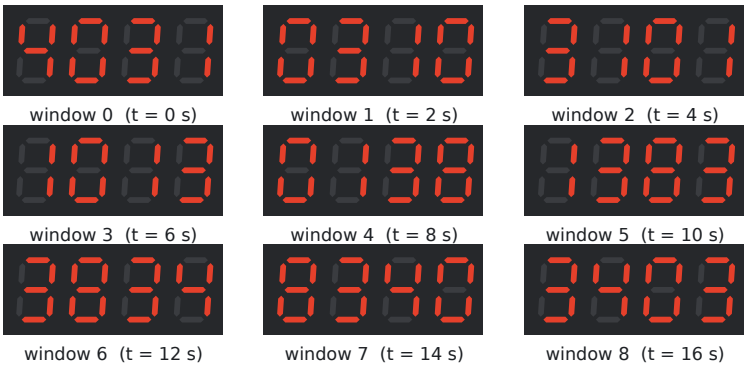


Figure 3. Part 5: the nine windows shown by the corrected design, one every 2 s.

7 Part 6 – Stopwatch

7.1 Design

Four modulo counters form the time `M SS.d`: tenths (mod 10), seconds (mod 10), tens of seconds (mod 6) and minutes (mod 10). All counters share the 10 Hz tick clock and are cascaded *synchronously*: each counter is enabled by the `done` output of the previous one, which is asserted while that counter is enabled and at its terminal count. The digits are multiplexed as in Part 3 with the tenths on the rightmost digit (Fig. 4).

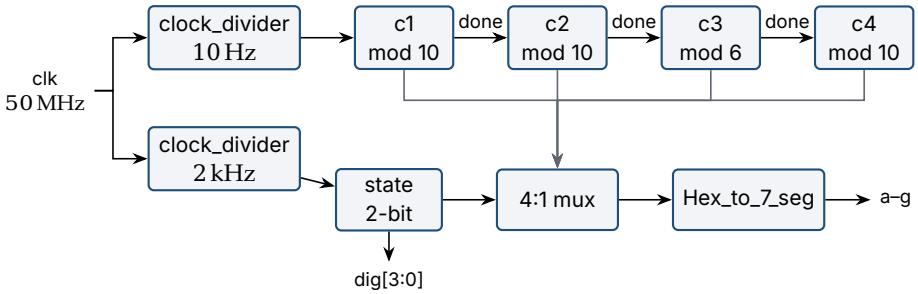


Figure 4. Stopwatch architecture. All counters are clocked by the 10 Hz tick; the `done` signals act as synchronous carry/enable.

Listing 1. Modulo counter with enable and carry (`mod_counter.v`).

```
always @(posedge clk)
    if (enable)
        number <= (number == N) ? 4'd0 : number + 4'd1;

assign done = enable && (number == N);   // next counter advances on this tick
```

7.2 Verification

With the tick shortened to 32 clock cycles, the testbench runs the stopwatch through a complete cycle of 6000 ticks (including the roll-over `9:59.9`→`0:00.0`) and after every tick reads the multiplexed display and compares it with the elapsed time: 6003/6003 readings correct. Fig. 5 shows the carry chain.

8 Bonus tasks

8.1 A – Adjustable speed

The tick divider is replaced by a divider whose half-period is a register: a rising edge on `button[1]` doubles it (slower) and on `button[2]` halves it (faster); edges are found by comparing each button with its value one clock earlier. Simulated: tick period 16→32 cycles after `button[1]`, then 8 after two presses of `button[2]`.

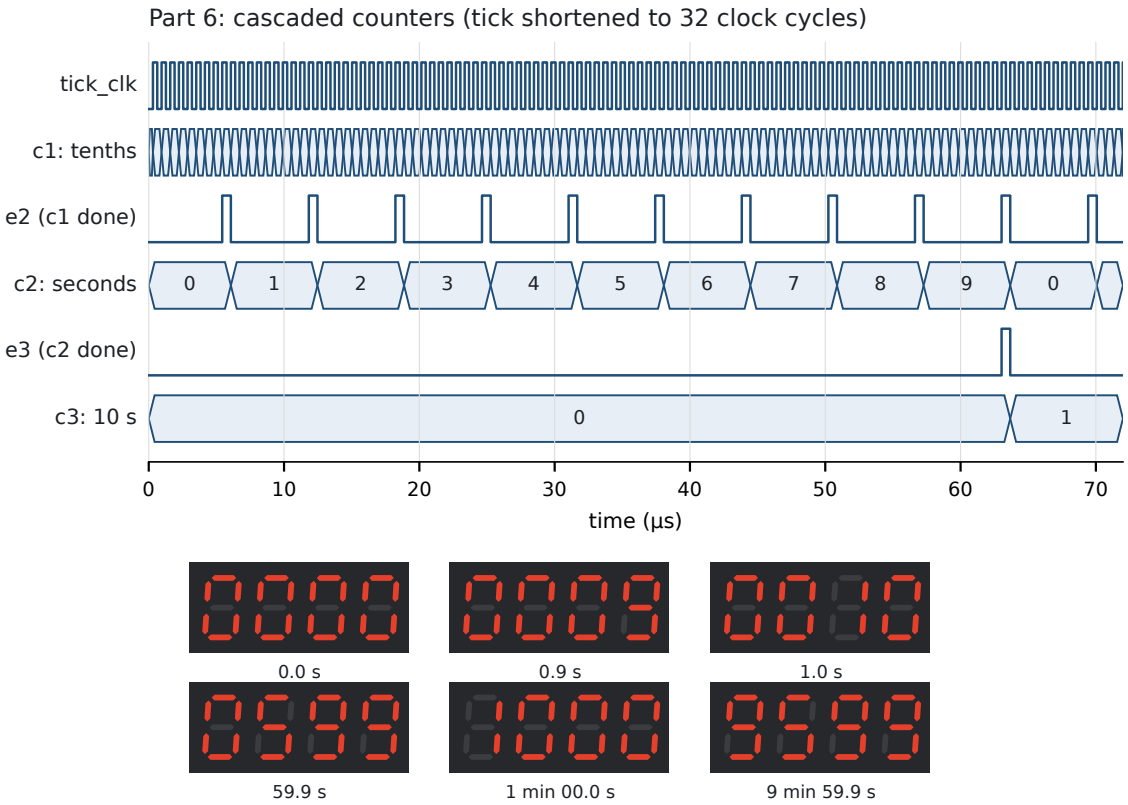


Figure 5. Part 6: c2 advances once per ten ticks (on `e2`), c3 once per hundred (on `e3`); bottom: display readings at selected times.

8.2 B – Start, stop and reset

`button[1]` clears and `button[2]` sets a `run` flag that enables the first counter (the stopwatch powers up stopped); `button[3]` synchronously clears all counters. Simulated sequence: power-up `0000`, counting after start, value held while stopped, `0000` after reset.

8.3 C – Snake

The outer segments of the four digits form a ring of 12 positions (top row left to right, the right side of digit 0, the bottom row right to left, the left side of digit 3). A 3-segment snake advances one position per 0.67 s: a combinational block ORs the masks of the head and the two following positions into a pattern per digit, and the patterns are multiplexed at 381 Hz. Timing comes from taps of a free-running counter (bits 16 and 24). The testbench records one full loop and checks that every frame has exactly three lit segments (Fig. 6).

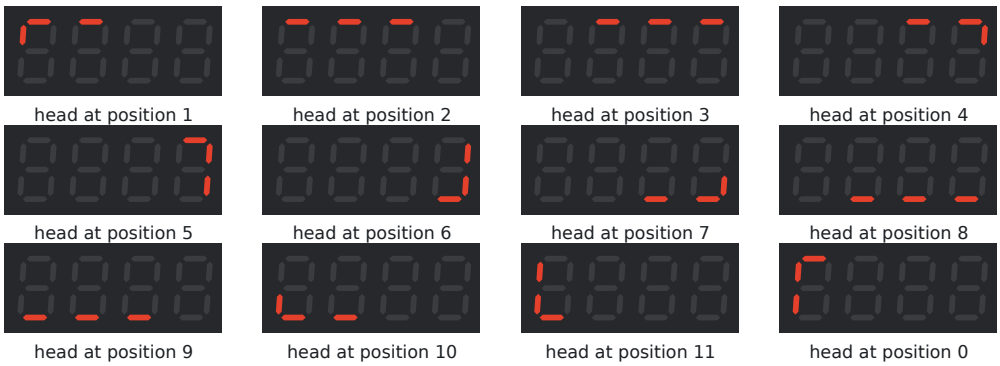


Figure 6. Bonus C: one loop of the snake reconstructed from the simulation.

In this design the digit enables are driven *active-low* (`1110` selects digit 0), whereas Parts 3–6 drive them active-high (`0001`). The cleaned version keeps the submitted behaviour behind a parameter `DIGIT_ACTIVE_LOW` (default 1); with the board's active-high enables it should be set to 0.

9 Implementation results

Table 2 is taken from the Vivado reports of the lab session (XC7Z010, 17 600 LUTs, 35 200 flip-flops). All designs meet the 20 ns clock constraint with a large margin.

Table 2. Vivado post-implementation results.

Design	LUTs	Flip-flops	WNS (ns)	Timing
Part 5 – scrolling student number	29	72	14.893	met
Part 6 – stopwatch	42	84	15.393	met
Bonus B – stopwatch with start/stop/reset	47	88	14.655	met
Bonus C – snake	116	48	17.608	met

Design note. The divided clocks are used directly as clocks of other registers. This works at these rates, but the static timing analysis only covers the 50 MHz domain unless the generated clocks are constrained. The usual alternative – keeping one clock and using one-cycle *enable* pulses – is the style used in the second ping-pong game of Experiment 6.

10 Changes made during this review

- **Part 2.** State 3 displayed `4` instead of `1`, so the sequence read 4 0 3 **4** 0 1 3 8 3. Corrected.
- **Part 5.** The leftmost digit (`dig = 1000`) was given the *last* nibble of the window, so every window appeared mirrored (e.g. `0310` shown as `0130`, confirmed by simulating the submitted code). The nibble order of the scan multiplexer was reversed.
- **All parts.** Dividers became parameters (so testbenches can shorten them), registers received initial values, unused modules were removed, and comments were rewritten in English (the original Persian comments had been saved as `?` characters and were not recoverable).

11 Verification summary

Testbench	What is checked	Result
`tb_show4dig`	all 256 DIP settings against a reference decoder	**PASS**
`tb_show_student_id`	displayed digit sequence	**PASS**
`tb_display_1234`	reconstructed image, one digit enabled at a time	**PASS**
`tb_moving_digit`	character and direction of every move	**PASS**
`tb_scrolling_student_id`	image of all 9 windows	**PASS**
`tb_stopwatch`	display after each of 6003 ticks incl. roll-over	**PASS**
`tb_stopwatch_speed_control`	tick period after the speed buttons	**PASS**
`tb_stopwatch_start_stop`	power-up, start, stop (hold), reset	**PASS**
`tb_snake_display`	one loop, three lit segments per frame	**PASS**

Reproduce with `scripts/run_sims.sh lab07-seven-segment` (Icarus Verilog 12).

12 Conclusion

Time-multiplexing lets four digits be driven by eleven pins as long as each digit is refreshed well above the flicker threshold; here a 2 kHz scan gives a 500 Hz refresh. Synchronous cascading through `done/enable` turns simple modulo counters into a multi-digit stopwatch, and the same structure accepts run control and speed control with a few registers. Simulating the displayed image – not just the internal signals – was what exposed the two display-order errors.